

Chapter 25

The Business Analyst Track

The business analyst on a data team: how the work divides between BA, data analyst, data scientist and data engineer, and how to specify the four things a data team is asked to build — a dashboard, a pipeline, a model, and a metric.

Draft chapter · v2

Contents

Chapter 25. The Business Analyst Track

Why this matters

In plain English

25.1 The business analyst on a data team

25.2 The software development life cycle, and where you sit in it

25.3 From a vague ask to a written requirement

25.4 Mapping the process before you change it

25.5 Business, functional, and non-functional requirements

25.6 The four things a data team is asked to build

25.7 The documents: BRD, FRD, and SRS

25.8 Use cases and user stories

25.9 Gap analysis

25.10 User acceptance testing, and why "it works" is not "the number is right"

25.11 Working with IT and vendors, and why domain knowledge decides everything

25.12 Finding, ranking, and specifying an automation

Common mistakes and how to spot them

In the real world: the report that passed every test

Tools

The project: map order-to-cash, and specify one data product

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 25. The Business Analyst Track

Part II – The Analyst

Chapter at a glance

You will learn to: say what a business analyst does on a data team, and how the work divides between the BA, the data analyst, the data scientist, and the data engineer · place that work inside each phase of the software development life cycle · turn a vague ask into a written, testable requirement · map a process as a flowchart, as a swimlane, and in BPMN, and document it as-is before proposing a to-be · tell business, functional, and non-functional requirements apart, including the non-functional ones specific to data: freshness, grain, completeness, and volume · write the requirement for each of the four things a data team is asked to build: a report or dashboard, a pipeline, a model, and a metric definition · choose between a BRD, an FRD, and an SRS, and know who reads each · write a use case, and a user story with acceptance criteria that work for data, including reconciliation and edge cases · run a gap analysis that ends in a requirement · explain what user acceptance testing is for a data product, and why "the system works" is not "the number is right" · work with an IT team or an outside vendor · find the automation opportunities in a process, rank them, and specify the one you pick.

Before you start: Chapter 3 (how a business runs on data, especially section 3.2's order journey and section 3.7's manual work) and Chapter 24 (turning an ask into a question, documenting business rules, and stakeholders). Chapter 23 (KPIs, and section 23.13 on defining a metric so two teams agree) helps, as does Chapter 16 (dashboards) and Chapter 20 (delivering reports).

Time needed: 5–7 hours, including the exercises and the project.

Tools: a pen and paper, or any free diagramming tool (draw.io, Lucidchart, or even PowerPoint) for the process maps. A shared document for the requirements. No software to install.

Practice data: none new. This chapter works on the Riverstone order that Chapter 3 followed from enquiry to cash, order 5001, and on the manual steps Chapter 3 listed. Every figure and date here is the one that chapter established.

Why this matters

Chapter 24 left you able to turn "make the report better" into a precise question and a memo that lands. That is most of what an analyst needs when the answer is a query.

This chapter is what happens when the answer is a **thing that has to be built**: a dashboard somebody will open every Monday, a pipeline that has to survive a late file, a model whose mistakes cost money, a metric that two departments have to agree on. Somebody has to work out what is actually needed, write it down precisely enough that it can be built, and stay involved until what appears is what was needed.

That somebody is a **business analyst**, and it is one of the ten roles Chapter 7 mapped. Chapter 7 puts it on the analytics track, next to the data analyst and the BI developer. This chapter is that role in full.

Three reasons to read it, and you may only care about one.

If the BA track is the job you want, this is the map. It is one of the most common entry points into a data career for people coming from operations, sales, finance, or support, because the first qualification is knowing how a business actually runs, which you may already have.

If you want to be a data analyst, data scientist, or data engineer, this is the half of your job that nobody teaches. Every dashboard you build starts as somebody's sentence. Every pipeline has a requirement behind it, written down or not, and the ones that are not written down are the ones you rebuild twice. Chapter 8's skills matrix marks requirements work as a "hidden skill" in data job descriptions, which is a polite way of saying it is expected and rarely listed.

And if neither, there is still the automation thread. Chapter 3 found six manual steps in one Riverstone order. Chapters 19 and 20 automated reports. Somebody has to decide *which* manual step is worth automating and write down what "done" means before anyone builds it. That is BA work, and doing it badly is how companies end up with automations nobody uses.

In plain English

Think of a house extension.

The person who wants it says, "we need more space". That is the ask, and it is not buildable. Somebody has to sit with the family and find out that what they actually need is a room where two children can do homework without hearing the television, that it has to be finished before the school year, that the budget is fixed, and that the door has to be wide enough for a wheelchair because the grandmother visits.

Then somebody draws the house as it is, because you cannot extend a wall you have not measured. Then somebody draws the house as it will be. The difference between the two drawings is the work.

Then it is written down precisely enough that a builder who was not in any of those conversations can build the right thing: dimensions, materials, where the plug sockets go, what "finished" means. And when it is built, the family walks through it and says whether it is what they needed, which is a different question from whether the builder followed the drawing.

A business analyst does all of that, and the house is usually a dashboard.

25.1 The business analyst on a data team

Four roles sit around most data work. They start from the same business question and diverge on what they produce.

	Business analyst	Data analyst	Data scientist	Data engineer
Starts from	a problem with how something works	a question about what happened	a question about what is likely	a need for data to arrive, reliably
Produces	a specification: process maps, requirements, acceptance criteria	an answer: a number, a chart, a memo	a model, and an estimate of how well it works	a pipeline, a table, a contract
Succeeds when	someone builds the right thing	someone makes a better decision	a decision gets made better than by rule of thumb	the data is there, on time, correct
Main risk	a precise specification of the wrong need	a correct answer to the wrong question	a strong model that serves no decision	a pipeline that is technically running and quietly wrong
Spends the day	interviewing, mapping, writing, in reviews	querying, cleaning, charting, writing	framing, feature work, training, evaluating	building, testing, monitoring, fixing

The boundary moves by company. In a small company one person does all four, and at Riverstone that person is currently Meera. In a bank they sit in four departments. What does not change is the shape of the BA's work:

1. **Elicit.** Find out what is actually needed, which is rarely the first thing said.
2. **Analyze.** Understand the current process well enough to know where the problem really is.
3. **Specify.** Write it down so it can be built and tested.
4. **Validate.** Check that what was built meets the need, not only the specification.

Those four verbs are the whole job. Everything else in this chapter is a technique for doing one of them well.

Why the other three roles need this chapter. A data analyst who cannot run step 1 builds the wrong dashboard. A data scientist who cannot run step 3 delivers a model nobody deploys, because nobody agreed what "good enough" meant. A data engineer who cannot run step 2 builds a pipeline on top of a source that a person quietly edits by hand every Friday. The four verbs are not a separate job. They are the part of every data job that happens before the code.

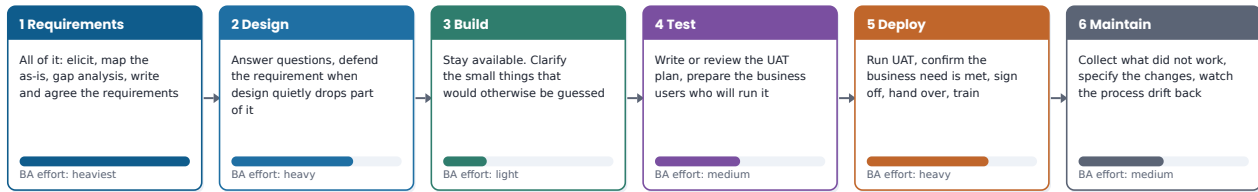
A warning that will save you a year. The most common mistake here is treating step 3 as the job and the other three as overhead. A beautifully written requirement for the wrong problem is worse than a rough note about the right one, because the rough note invites a conversation and the polished document ends it.

25.2 The software development life cycle, and where you sit in it

Most things a BA specifies get built by somebody else, and that building follows a shape called the **software development life cycle**, or SDLC. The names of the phases vary; the sequence does not. A dashboard, a pipeline, and a model all go through it, in a lighter form than a banking system but the same shape.

The software development life cycle, and what the business analyst does in each phase

The requirements phase is not the job. It is the first sixth of it.



Sign-off happens at the end of phase 1. Most of the damage in this job happens in phases 2 to 6.

Figure 25.1 – The BA's work does not stop when the requirements are signed off. It thins out and changes shape.

Phase	What happens	What the BA does
Requirements	the need is established and written down	all of it: elicitation, process mapping, gap analysis, writing the requirements and getting them agreed
Design	technical people decide how to build it	answers questions, defends the requirement when the design quietly drops part of it, records the decisions that change scope
Build	it gets built	stays available, clarifies, handles the small questions that would otherwise be guessed at
Test	QA checks the system behaves as specified	writes or reviews the UAT plan, prepares the business users who will run it
Deploy	it goes live	runs UAT, confirms the business need is met, signs off or does not, plans the handover and the training
Maintain	it runs, and changes	collects what did not work, specifies the changes, watches the process drift back toward its old shape

Waterfall and Agile. In a **waterfall** project the phases run once, in order, and the requirements are signed off before the build starts. In an **Agile** project the same six activities repeat every two or three weeks on a small slice of the work, so the requirements arrive as a stream of user stories rather than one document. Most data work is closer to Agile, because a dashboard is worth showing after a week. Chapter 26 covers how Agile is actually run, with Scrum, Kanban, and Jira. What matters here is that the four verbs do not change. Agile changes the batch size, not the job.

The trap in both. In waterfall, the BA disappears after sign-off and the system drifts. In Agile, the stories get so small that nobody is holding the shape of the whole thing. The defense against both is the same: keep the process map and the business need visible after the requirements are written.

25.3 From a vague ask to a written requirement

Chapter 24 taught the first move: when someone says "make the report better", find out what they would do differently with a better one. This section is what happens next, when the answer turns out to need something built.

The ask: "We need a way to see which customers haven't ordered recently."

That sentence is not a requirement. It has at least four holes in it, and your first job is to find them before writing anything down.

1. **Who is "we", and what will they do with it?** The answer decides everything downstream, including whether this is a dashboard, an alert, or a column on a list somebody already reads.
2. **What does "recently" mean?** Thirty days? Ninety? Longer than that customer's own usual gap, which is a different and much better rule?
3. **What does "haven't ordered" mean?** No order placed, no order delivered, or no order paid for? A cancelled order counts how? This is a **grain and definition** question, and it is where data requirements are won or lost.
4. **What happens when the answer is wrong?** If a customer is listed who did order, what is the cost? If one is missed, what is the cost? The answers set the tolerance, and for a model they set the whole evaluation.

Ask those four and you often find the ask was not one requirement at all. Sometimes it is several, wearing one sentence as a disguise. Sometimes it is one requirement that nobody needed, which is the story at the end of this chapter.

The discipline, in five moves:

1. **Ask for a specific example of the pain.** "Tell me about the last time this cost you something." Abstractions hide the requirement; incidents contain it.
2. **Ask about the exception path, not the happy path.** What happens when the file is late, when the customer disputes, when two systems disagree. For data work this is most of the requirement: late data, missing data, duplicate data, and data that arrives changed.
3. **Ask how often.** Frequency decides whether the work is worth doing, and you need it long before anyone asks for a business case.
4. **Restate it back in your own words and let them correct you.** Being corrected is the point. It is much cheaper here than in UAT.
5. **Write it as something testable.** If you cannot describe how you would prove the requirement was met, it is not a requirement yet.

25.4 Mapping the process before you change it

You cannot improve a process you have not drawn. And you cannot draw one accurately from your desk, because the written procedure and the real process are different documents.

A flowchart shows what happens

The simplest map is a **flowchart**: boxes for steps, diamonds for decisions, arrows for sequence. Chapter 3 followed one Riverstone order through ten of them, from Rakesh's enquiry on 22 October 2025 to the January report on 3 February 2026.

A flowchart is enough when one team owns the whole process. Riverstone's order-to-cash is not that, and neither is any process that produces data worth analyzing.

A swimlane shows whose job it is

A **swimlane diagram** is a flowchart with one lane per role or department. Every step sits in the lane of whoever is responsible for it, which makes something visible that a plain flowchart hides completely: the **handoffs**, the points where work crosses from one lane to another.

Riverstone order 5001, order-to-cash, by the team that owns each step

Five of the nine transitions cross a lane. Those crossings are where the delay and the errors collect.

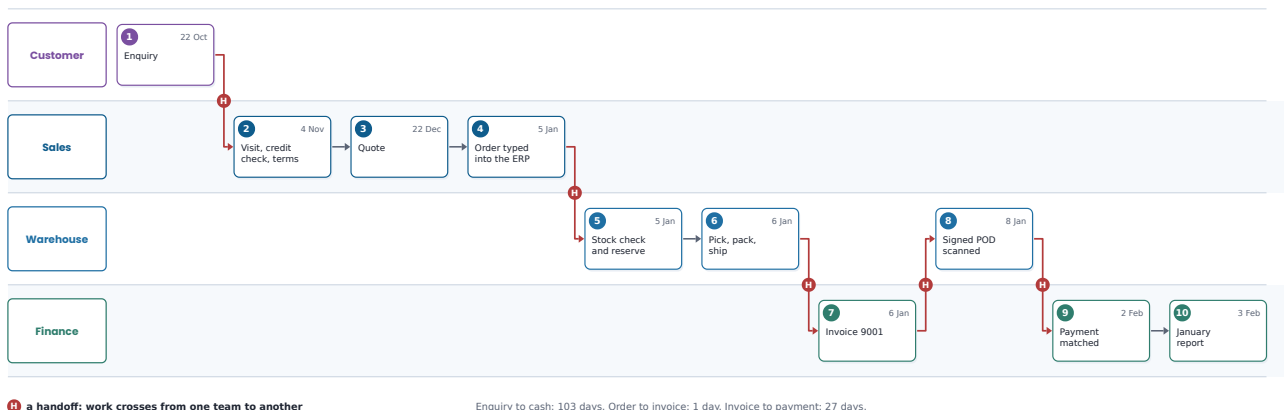


Figure 25.2 – The same ten steps as Figure 3.2, now showing who owns each. Five of the nine transitions cross a lane.

Each step goes in the lane of whoever does the work, even where two parties are involved. Rakesh asks for prices at step 3, but it is Neha who builds and sends the quote, so step 3 sits in the sales lane.

Lane	Steps it owns
Customer	1 enquiry
Sales	2 visit and terms, 3 quote, 4 order typed into the ERP
Warehouse	5 stock check and reserve, 6 pick, pack, ship, 8 signed proof of delivery scanned
Finance	7 invoice, 9 payment matched, 10 January report

Now read Figure 25.2 again and notice where the time went. Chapter 3 measured the whole journey at 103 days, but only 28 days from order to cash, and only 1 day from order to invoice. The long waits are on the customer's side of the lane boundary, which is a different problem from the one inside the building. **The delays a project can fix are almost always at the crossings between lanes, not inside them.**

There is a second reason this matters to a data person specifically. **Every handoff is where a data quality problem is born.** Step 4 is where an order is re-typed, which is where the wrong product code enters the data you will later clean in Chapter 14. Step 8 is where a proof of delivery becomes an email attachment, which is why delivery dates are missing. Step 9 is where a bank line is matched by eye, which is why some payments sit against the wrong invoice. When you map a process and find the handoffs, you have also found the columns you will not be able to trust.

BPMN makes the map unambiguous

BPMN, Business Process Model and Notation, is a standard symbol set: specific shapes for events, tasks, gateways, and messages passed between participants. A circle is an event, a rounded box is a task, a diamond is a gateway, a dashed arrow is a message crossing between participants.

You do not need all of BPMN. You need to know that it exists and why it is used: when a process map leaves the room, to an IT team or an outside vendor who was not in any of the interviews, a standard notation means they read it the way you drew it. A home-made diagram with your own symbols gets interpreted, and interpretation is where requirements die.

As-is before to-be

Two maps, in this order, always.

The **as-is** map documents the process exactly as it runs today, including the workarounds. Riverstone's as-is includes Neha phoning the warehouse because the ERP's stock figures are a day old, and somebody matching a bank line that reads "SHARMA HW JAN" to invoice 9001 by eye.

The **to-be** map documents the proposed process after the change.

Skipping the as-is is the most common shortcut in this work and it fails the same way every time: you propose an improvement to a process that does not exist, the workaround you never documented turns out to be load-bearing, and the new system breaks something nobody told you about because nobody was asked.

There is a second reason, and experienced BAs will tell you it is the bigger one. **The as-is map is frequently the most valuable thing you produce, before any change is made at all.** People who run a process day to day usually know their own step and the two next to it. Nobody has seen the whole thing. Put the map on a wall and the room starts talking: *why do we do that twice? who asked for that approval? does anyone read that email?*

25.5 Business, functional, and non-functional requirements

Three words that get used interchangeably and should not be.

A **business requirement** says what the organization needs and why. It contains no technology. *Riverstone needs to invoice customers within one working day of shipment, because the current one to two day delay adds directly to the time between shipping goods and collecting cash.*

A **functional requirement** says what the system must do. *When an order's status changes to Shipped, the system shall generate the invoice for that order.*

A **non-functional requirement** says how well: speed, volume, availability, security, accuracy.

	Answers	Written by	Read by
Business	why are we doing this	BA, with the sponsor	executives, finance
Functional	what must it do	BA	developers, testers
Non-functional	how well must it do it	BA, with technical input	developers, architects, operations

The non-functional requirements that are specific to data

Generic non-functional requirements are about speed and uptime. Data work has its own set, and these are the ones that get dropped and then cause an incident:

Type	The question it answers	An example
Freshness	how old may the data be when someone reads it?	the dashboard shall show data no more than two hours old during working hours
Grain	one row is one what?	one row per order line, not per order
Completeness	what share of rows may be missing or unknown?	fewer than 0.5% of orders may have no sales rep
Accuracy and reconciliation	what must it agree with, and how closely?	monthly revenue shall reconcile to the ERP's own figure to the rupee
Timeliness	by when must it be ready?	the daily file shall be loaded by 07:00 IST
History	how far back, and what happens when the past changes?	three years retained; a corrected order updates history and is flagged
Access	who may see which rows?	a regional rep sees only their own region
Volume	how much, growing how fast?	210,000 order lines today, about 15% growth a year

Non-functional requirements are the ones that get dropped, because they are invisible when the thing is demonstrated with five rows of test data and expensive to add once it is built. A dashboard that takes four minutes to open is technically working and practically abandoned. A pipeline with no stated freshness is a pipeline nobody can tell is late. Write the numbers down in the first draft.

Chapter 60 takes these to architecture scale, where non-functional requirements decide the shape of the whole system. The precision you learn here is the precision that chapter assumes.

25.6 The four things a data team is asked to build

Almost every request that reaches a data team turns into one of four things. Each has its own questions that must be answered before anybody starts, and each has a way of going wrong that the other three do not share.

The four things a data team is asked to build, and what each requirement must pin down

Ask the wrong set of questions and you will build something that works and nobody uses.

Report or dashboard	Pipeline	Model	Metric definition
- Who opens it, how often? - What do they do next? - What is one row? - Default date range? - Who may see which rows? HOW IT FAILS Fails by answering a question the reader already knew	- What is the source of truth? - What happens when it is late? - What happens when the past changes? - Rerun twice: same result? - Who is alerted, and how fast? HOW IT FAILS Fails by running green and being quietly wrong	- What decision does it serve? - What does each mistake cost? - What is it compared against? - Who is never scored? - What is written out with the score? HOW IT FAILS Fails by being accurate while nobody changes what they do	- What exactly is counted? - What is excluded? - Which teams must agree? - Where does the definition live? - How is a change announced? HOW IT FAILS Fails by living inside a query, reinvented by every new report

Figure 25.3 — Ask the wrong set of questions and you will build something that works and nobody uses.

A report or dashboard

BR-01. Key accounts sales managers need to see which accounts have gone quiet, so they can decide which one to spend a call on this week.

FR-01. The dashboard shall list accounts whose last order is more than 1.5 times their own average gap between orders. **FR-02.** Each row shall show the account's last twelve months' revenue and the date of its last order. **FR-03.** A user shall see only accounts where they are the assigned rep, except that a sales head sees all.

NFR-01. Data shall be no more than one working day old. **NFR-02.** The dashboard shall open in under five seconds with three years of history loaded. **NFR-03.** Revenue shown shall reconcile to the ERP's monthly figure to the rupee.

The questions to ask before writing any of that: **who opens it, how often, and what do they do next?** A dashboard nobody acts on is a cost with a nice color scheme. Then: what is one row? What date range is the default? What happens when a number looks wrong, and who do they ask?

How dashboards fail: they answer a question the reader already knew the answer to. That is the story at the end of this chapter.

A pipeline

BR-02. Analysis of sales needs order data in the warehouse each morning, because the team's questions are asked at the 9 a.m. meeting.

FR-04. The pipeline shall load the previous day's orders, order lines, and invoices from the ERP.

FR-05. Where a row already exists, it shall be updated rather than duplicated. **FR-06.** A row that fails validation shall be written to a rejects table with the reason, and shall not be loaded. **FR-07.** The run shall be idempotent: running it twice for the same day shall leave the same result.

NFR-04. Loaded and validated by 07:00 IST on working days. **NFR-05.** A failed run shall alert a named owner within fifteen minutes. **NFR-06.** A backfill of any past day shall be possible without deleting other days.

The questions: **what is the source of truth, what happens when it is late, and what happens when the past changes?** A customer's city gets corrected in March for an order placed in January. Does your table change? That single question separates a pipeline that survives from one that quietly disagrees with the ERP forever. Chapters 45 and 46 build exactly this; your job is to specify it before they do.

How pipelines fail: they keep running and stop being right. Nobody notices, because a green tick is not the same as a correct number.

A model

BR-03. Riverstone needs to know which accounts are likely to stop ordering, early enough to do something about it.

FR-08. The model shall produce, for each active account, a probability of no order in the next ninety days. **FR-09.** The output shall be written to a table refreshed weekly, with the score date and the model version. **FR-10.** Each scored account shall carry the three features that contributed most to its score.

NFR-07. The model shall beat the current rule of thumb, "no order in ninety days", measured on accounts held back from training. **NFR-08.** A score shall never be produced for an account with fewer than three historical orders.

The questions: **what decision does this serve, what is the cost of each kind of mistake, and what is it being compared against?** A model with no baseline is unfalsifiable. A model whose false positives and false negatives cost the same amount is rare, and if you have not asked, you have not specified it. Chapters 36 and 39 teach the measurement; this is where you agree what "good enough" means, in advance, with the person who will live with it.

How models fail: they are accurate and nobody changes what they do. That is a requirements failure, not a modeling one.

A metric definition

The smallest of the four and the one that causes the most arguments.

BR-04. Sales and finance shall use one definition of monthly revenue, because the two teams currently report different numbers for the same month.

FR-11. Monthly revenue shall be the sum of invoiced line value after discount, for invoices dated in that month, excluding cancelled orders and excluding tax. **FR-12.** The definition shall be recorded once and referenced by every report that uses it. **FR-13.** A change to the definition shall be versioned, dated, and announced before it takes effect.

Chapter 23 section 13 showed what happens without this: two teams, one word, three numbers. The BA's contribution is not the arithmetic. It is getting both teams to agree to the same sentence, and then writing it somewhere both of them will find it.

How metrics fail: the definition lives in somebody's query, so every new report reinvents it slightly differently.

25.7 The documents: BRD, FRD, and SRS

Three documents, three audiences, three levels of detail. The names vary by company, and some companies collapse all three into one. The distinction still helps, because the mistake it prevents is writing at the wrong level for the person reading.

Document	Full name	Answers	Written for	Typical contents
BRD	Business Requirements Document	<i>why</i> , and what outcome the business needs	sponsors, executives, finance	the problem, the business case, scope and what is out of scope, the business requirements, success measures, assumptions, risks
FRD	Functional Requirements Document	<i>what</i> the system must do	developers, testers, the BA	numbered functional requirements, process flows, business rules, data definitions, screen and report descriptions
SRS	Software Requirements Specification	<i>what and how well</i> , in full technical detail	developers, architects, QA	everything in an FRD plus interfaces, non-functional requirements, data models, constraints, error handling

A rough rule: the **BRD** should be readable by someone who will never log in. The **FRD** should be detailed enough that two developers reading it build the same thing. The **SRS** should be detailed enough that a vendor could build it without talking to you, which is why vendor projects live or die on it.

For a dashboard inside your own company, one page with BR, FR, and NFR sections is usually the whole documentation set, and the three-document structure is what you scale up to when the thing is bigger or the builder is further away.

Requirements get numbers, and the numbers matter. BR-01, FR-07, NFR-03. A number lets you trace a line of code back to a business reason, and lets you notice that FR-12 was quietly dropped in design. That trail is a **requirements traceability matrix**: one row per requirement, with columns for the business requirement it serves, the design element that implements it, the test that proves it, and its status. In a small project it is a spreadsheet, and Chapter 11's skills are enough to maintain one.

25.8 Use cases and user stories

Both describe what somebody does with the system. They differ in size and in what they are for.

A use case

A **use case** describes one complete interaction, in steps, including what happens when it goes wrong.

UC-04: Match a payment to an invoice **Actor:** finance assistant **Precondition:** a bank statement line has been imported and is unmatched. **Main flow:**

1. The assistant opens the unmatched payments list.
2. The system shows, for each unmatched line, the customers whose outstanding invoices match the amount.
3. The assistant selects the correct invoice.
4. The system records the payment against the invoice and marks the line matched. **Alternative flow A, partial payment:** at step 3 the assistant records an amount smaller than the invoice total; the system records a part payment and leaves the invoice open for the remainder. **Alternative flow B, no candidate:** at step 2 the system finds no matching invoice; the assistant marks the line for investigation with a note. **Postcondition:** the bank line is matched, part-matched, or flagged, and never silently ignored.

The two alternative flows are most of the value. The main flow was obvious.

A user story

A **user story** is one sentence in a fixed shape, plus the conditions that decide when it is done. It is sized to be built in a few days.

A user story, and the acceptance criteria that make it testable

Without criteria a story cannot be tested, so it can never be finished.

As a	sales rep	who wants it
I want	to see which of my accounts are at risk of not ordering again	what they want
so that	I can call them before they go quiet	why it is worth building
<i>The "so that" clause is the one people drop, and the one that stops you building something correct and useless.</i>		

AC-1	Given I am a logged-in sales rep, when I open the at-risk list, then I see only accounts where I am the rep. <i>scope</i>
AC-2	Given an account whose last order was more than 1.5 times its own average gap, when the list is generated, then it appears. <i>the actual rule, not the word "recently"</i>
AC-3	Given an account with no orders at all, when the list is generated, then it does not appear. <i>the edge case</i>
AC-4	Given the list is open, when I sort by value at risk, then accounts order by last twelve months' revenue, highest first. <i>behavior the user can check</i>

Figure 25.4 – The three clauses of a story, and four criteria that make it testable. AC-3 is the edge case.

As a sales rep, I want to see which of my accounts are at risk of not ordering again, **so that** I can call them before they go quiet.

The shape forces three things into the open: who wants it, what they want, and why. The "so that" clause is the one people drop, and it is the one that prevents building something correct and useless.

A story without **acceptance criteria** cannot be tested, so it cannot be finished. The common form is Given, When, Then:

AC-1 Given I am a logged-in sales rep, *when* I open the at-risk list, *then* I see only accounts where I am the assigned rep. **AC-2** Given an account whose last order was more than 1.5 times its own average gap between orders, *when* the list is generated, *then* that account appears on it. **AC-3** Given an account with no orders at all, *when* the list is generated, *then* it does not appear, because there is no gap to compare against. **AC-4** Given the list is open, *when* I sort by value at risk, *then* accounts are ordered by their last twelve months' revenue, highest first.

Three things to copy. **AC-2 contains the actual rule**, not the word "recently". **AC-3 is an edge case**, and a story with no edge case has not been thought about. And every criterion could be run by hand by a person who says yes or no, which is what makes them usable later as acceptance tests.

Acceptance criteria for data have a fourth kind, and it is the one people new to this work miss. Alongside scope, rule, and edge case, write a **reconciliation criterion**:

AC-5 Given the dashboard is refreshed, *when* its total revenue for any complete month is compared with the ERP's figure for that month, *then* the two agree to the rupee.

That sentence is the difference between a dashboard people trust and a dashboard people check against a spreadsheet before every meeting.

	Use case	User story
Size	a whole interaction	a slice of one
Detail	steps, alternatives, pre- and postconditions	one sentence plus criteria
Suits	waterfall, vendor contracts, complex exception handling	Agile delivery, incremental build
Risk	a specification nobody reads to the end	loses the shape of the whole process

They are not rivals. A common pattern is one use case for the process and several user stories to build it.

25.9 Gap analysis

A **gap analysis** compares the as-is state against the to-be state and names the specific differences that have to be closed. Each gap becomes a candidate requirement.

Done badly it produces a list of observations. Done well every row ends in something buildable, and names the **root cause** rather than the symptom.

One row of a gap analysis, from Riverstone's invoicing step

A row is not finished until it produces a numbered requirement.

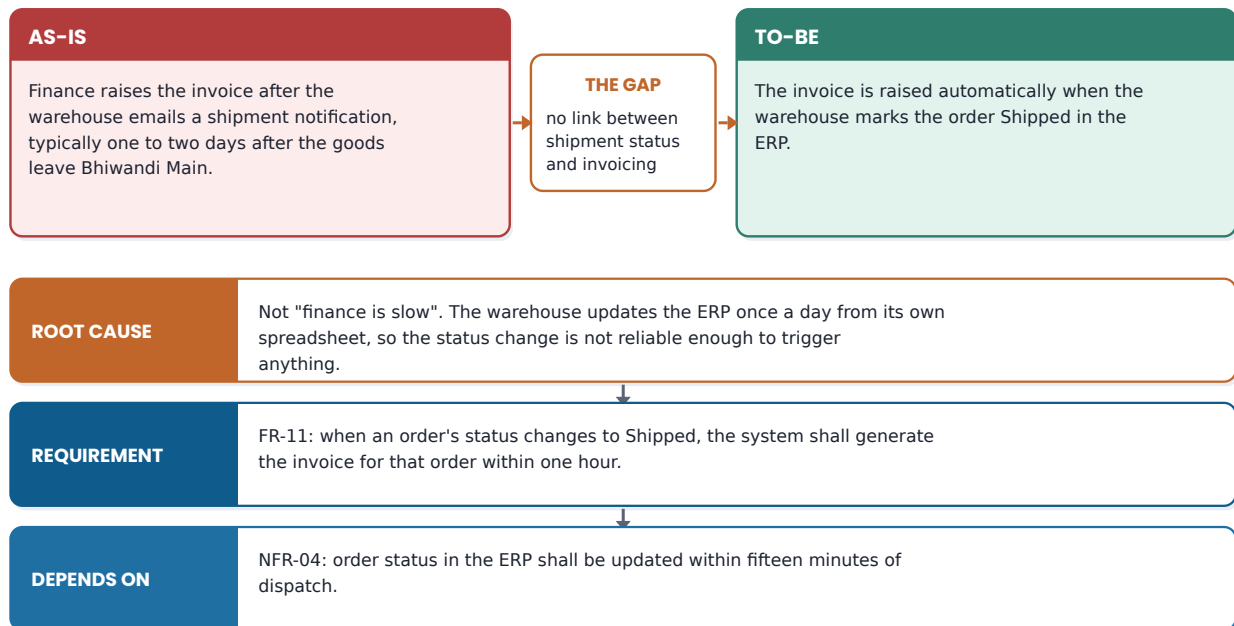


Figure 25.5 – A gap analysis row is only finished when it produces a requirement.

As-is. Finance raises the invoice after the warehouse emails a shipment notification, typically one to two days after the goods actually leave Bhiwandi Main. **To-be.** The invoice is raised automatically when the warehouse marks the order Shipped in the ERP. **Gap.** There is no link between the warehouse's shipment status and invoicing. The handoff is an email read by a person. **Root cause.** Not "finance is slow". The warehouse updates the ERP once a day from its own spreadsheet, so the status change is not reliable enough to trigger anything. That is the thing to fix first. **Resulting requirement.** FR-14: when an order's status changes to Shipped, the system shall generate the invoice for that order within one hour. **Depends on.** NFR-09: order status in the ERP shall be updated within fifteen minutes of dispatch.

The root-cause line is what separates a useful gap analysis from a complaint. Without it, Riverstone would have built automatic invoicing on top of a status field that updates once a day, and invoices would have gone out a day late anyway, automatically.

Ranking the gaps. One analysis produces ten of these and you cannot do ten. Score each on what it saves and what it costs:

Gap	People affected	Time saved per month	Error risk removed	Build effort	Rank
Invoice on shipment status	finance, 1 person	~8 hours	invoices raised late, cash delayed	medium	1
Stock figures live, not daily	sales, 4 people	~6 hours	stock promised that is already sold	large	2
Order re-keyed from email	sales, 4 people	~17 hours	wrong quantity, wrong code, lost discount	medium	3

Those hour figures are not facts about a database. They are what the BA collected by asking the people who do the work, and they should be labeled that way in the document. A number you gathered in an interview is evidence; a number you invented to make a case is the thing this book has spent twenty-four chapters teaching you not to do.

25.10 User acceptance testing, and why "it works" is not "the number is right"

User acceptance testing, UAT, is the final validation: real business users checking whether the delivered thing meets the original business need.

The distinction that matters:

- **QA** verifies the system works as specified.
- **UAT** verifies the specification was right.

A system can pass one hundred percent of its QA test cases and fail UAT, because the test cases were written from a requirement that turned out not to capture what the business needed. QA cannot catch that, and neither can the developers, because both are working from the same document. Only the people with the original need can judge it.

For a data product, UAT has a second job that QA structurally cannot do: checking whether the numbers are right. A dashboard can pass every functional test, load in two seconds, respect every access rule, and show a revenue figure that is four lakh rupees short because cancelled orders were included. QA has no way to know. The finance manager who has been closing the month for six years knows in four seconds.

So a data UAT plan has two halves:

1. **Does it do what we said?** The acceptance criteria, run by a user.
2. **Does it agree with what we already know?** Reconcile against a source the business already trusts: last month's closed figures, the ERP's own report, a spreadsheet somebody has kept for years. Reconcile at least one complete period, at more than one level of aggregation. A total that matches while the breakdown does not is a real and common failure.

Who runs it. Real users doing their real work, not the BA demonstrating the system to them. If the BA drives the mouse, the test measures the BA.

Who signs off. The business stakeholder who owns the requirement, not IT and not the project manager. That signature is the moment accountability transfers, and it is worth treating as a decision rather than a formality.

What a UAT test case looks like. Written in the user's language, from their workflow, checking a business outcome. Well-written acceptance criteria are already UAT test cases: AC-2 and AC-5 in section 25.8 can be handed to a user as they stand.

When UAT fails close to go-live, which it does, triage rather than panic. Is it a defect, which gets fixed and retested? A missed requirement, which needs an honest impact assessment and possibly a delayed date? Or a misunderstanding of scope, which gets clarified and may be deferred? The temptation is to launch anyway and fix it afterward. Sometimes that is right. It has to be weighed against the real cost of the specific gap found, not waved through as normally acceptable and not treated as an automatic blocker.

25.11 Working with IT and vendors, and why domain knowledge decides everything

Most of what a BA specifies is built by people who were not in the room where the need was explained.

An internal IT or data team shares your company and your context. The risk is drift: the requirement gets refined in design meetings you were not in, and a small technical decision quietly removes something the business needed. The defense is presence. Go to the design reviews. When a decision changes what the business will get, say so at the time and write it down.

An outside vendor shares neither. The risk is contractual: anything not written down is out of scope, and you will be told so. This is where an SRS earns its cost, where BPMN earns its formality, and where non-functional requirements matter most, because "the dashboard opens in under five seconds with three years loaded" is either in the document or it is a change request.

Both share one rule. **Every decision that changes scope gets written down, dated, and agreed by the person who owns the outcome.** Not because anyone expects a dispute, but because six months later nobody remembers why the system does what it does, and your notes are the only record.

Domain knowledge is what makes any of this work. A BA who knows that Riverstone's Kolkata office loads orders through a spreadsheet upload, that the warehouse's stock figures come from a shadow spreadsheet, and that Plant 1 at Taloja makes to stock while other products are made to order, asks different questions from one who does not. They spot what is missing from the process map. They know which exception is rare and which one happens every Friday.

This is why the BA track is a common entry point for people already inside a business. Chapter 8 makes the argument in full: somebody who has spent four years in sales operations starts this job with the hardest part already done, and can learn the templates in a month. If you are coming from outside a domain, the substitutes are patience and questions. Sit with the people who do the work. Watch the process rather than reading the procedure. Ask what they do when it goes wrong, and then ask how often that is.

25.12 Finding, ranking, and specifying an automation

This is the section that connects the BA track to the automation thread running through this book, from Chapter 19's spreadsheet macros to Chapter 63's governance.

Finding it

Chapter 3 section 3.7 gave you the question: at every step of the map, did a person copy, re-type, check, or carry data by hand? Six of Riverstone's ten steps answered yes. The named patterns are worth memorizing, because you will see them everywhere, and because each one is also a data quality problem waiting to be found in Chapter 14:

- **Re-keying:** typing data that already exists somewhere else.
- **Copy-paste integration:** moving data between systems by copying it.
- **Emailing files around:** the data stops updating the moment it is attached.
- **Manual matching**, or reconciliation: pairing two sets of records by eye.
- **Shadow systems:** personal spreadsheets holding data the official system should hold.

Ranking it

Not every manual step should be automated. Rank candidates on four things, and be honest about the fourth:

1. **Frequency.** How many times a month does this happen?
2. **Time per occurrence.** How long does it take, measured rather than guessed?
3. **Cost of the errors it causes.** Not only the time, the consequence.
4. **Effort and risk to automate**, including what happens when the automation itself fails.

Chapter 3's illustration is the shape of the arithmetic: forty emailed orders a week at about six minutes each is four hours a week, roughly two hundred hours a year. That is the kind of number that decides whether a project happens.

The steps that should stay manual are the ones where a person's judgment is the point. Approving an unusual discount, deciding whether a disputed delivery gets credited, and reading a customer's tone in an email are not re-keying. Chapter 58 makes this case with real numbers: an email-intake pipeline that was right 88% of the time was still not safe to run unattended, because of what the other 12% did quietly.

Specifying it

The requirement for an automation is written like any other, with three additions specific to automating something a person used to do.

Take Riverstone's invoice matching, the step where a finance assistant reads a bank statement line and works out which invoice it pays.

BR-05. Riverstone needs payments matched to invoices without a person reading each bank line, because manual matching delays the cash position and produces customers being chased for money they have already paid.

FR-15. The system shall compare each imported bank statement line against open invoices for that customer, and record a match where the amount and the customer both agree.

FR-16. Where the amount differs from an open invoice by more than the tolerance in BR-06, the system shall flag the line for review by a named person and shall not record a match.

FR-17. The system shall record, for every automatic match, the rule that produced it and the time it was made, so that any match can be explained afterward.

NFR-10. The system shall process a day's bank file within fifteen minutes of receiving it.

The three additions:

- **A tolerance.** Exact matching and close-enough matching are different requirements, and close-enough is much harder. Never let "match" stay undefined.
- **An exception path with a named owner.** Every automation needs a route for the cases it cannot handle, going to a specific person and not to a queue nobody reads.
- **An audit trail.** When an automation makes a decision, somebody will eventually need to know why. FR-17 is what makes that possible, and it is the requirement most often left out.

And one question before any of it: **how often does the exception happen?** If nine out of ten bank lines match cleanly, automation removes most of the work. If half of them need judgment, you are building a system to handle the clean half while the person still has to read the file. That number decides whether the project is worth doing, and you get it by asking, early, before anybody has fallen in love with the idea.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Writing the vague ask down as the requirement	nobody can say what "done" means	ask for a specific example of the pain, then restate it as something testable
Never asking what the reader will do differently	a dashboard that is correct and unopened	the "so that" clause is a requirement, not a courtesy
Leaving the grain undefined	two people count the same thing and get different numbers	say what one row is, in writing, before anything is built
No freshness requirement	nobody can tell whether the data is late	put a time and a deadline in the first draft
No reconciliation criterion	every meeting starts by checking the dashboard against a spreadsheet	name what it must agree with, and to what tolerance
A model with no baseline	"84% accurate" with nothing to compare it against	write the rule of thumb it must beat into the requirement

Mistake	Symptom	Fix
Skipping the as-is map	the change breaks a workaround nobody mentioned	document the process as it really runs, however obvious it seems
Mapping only the happy path	exception handling gets built last, badly, or not at all	map every branch, including the rejections and the late file
A user story with no acceptance criteria	the story cannot be tested, so it is never finished	Given/When/Then, including at least one edge case
Treating BRD, FRD, and SRS as interchangeable	the wrong level of detail for the person reading	pick the document by its audience
A gap analysis that stops at the observation	a list of complaints, no requirements	every row ends in a numbered requirement
Naming the symptom as the root cause	the automation gets built on a broken foundation	ask why one more time than feels necessary
Disappearing after sign-off	the delivered thing has quietly drifted from the need	stay through design, build, and UAT
Treating UAT as a repeat of QA	the specification is never challenged, and the numbers are never reconciled	real users, their own work, and one complete period reconciled
Automating a step whose point is judgment	an automation people route around	rank on frequency, time, error cost, <i>and</i> whether judgment is the work
An automation with no exception route	failures go silently into a queue nobody reads	every automation needs a named owner for what it cannot handle
Agreeing a scope change in a corridor	six months later nobody knows why the system does that	write it down, date it, get it agreed by whoever owns the outcome

In the real world: the report that passed every test

Riverstone approved a small project in March 2026: a dashboard showing accounts at risk of going quiet. Vikram Singh, the key accounts sales manager, had asked for it twice.

Ayesha Qureshi, who had moved into a business analyst role from the customer support desk eighteen months earlier, took the requirement. She did the work properly by most measures. She interviewed Vikram. She wrote the business requirement, four functional requirements, and two non-functional ones. She drew the as-is: a rep noticing, usually late, that a customer had gone quiet. She wrote acceptance criteria with an edge case in them. QA tested the build against the specification and every test passed. The numbers reconciled to the ERP to the rupee.

UAT failed in eleven minutes.

Vikram opened the dashboard, scrolled, and said: "I know all of these. Coastal went quiet in January, I know why, their kitchen is being rebuilt. What I don't know is what to do about Northgate."

The specification was correct. The requirement was wrong. Ayesha had asked what Vikram wanted to see and had written down the answer. She had not asked the question from section 25.3 that would have caught it: *what will you do differently when you have this?* The answer, had she asked, was not

"know which accounts are quiet". Reps already knew that. It was "decide which quiet account to spend Thursday afternoon on", and that needs a reason and a value, not a list of names.

The fix took nine days and was mostly subtraction. The dashboard kept its list and added two columns: the value of the account's last twelve months, and which of three reasons the drop matched, seasonal, a lost tender, or a service complaint logged in the last quarter. The version Vikram now uses is shorter than the one that passed QA.

Three things are worth taking from this. **UAT did its job**: a build passed every technical test and every reconciliation, and still failed, and the only person who could see it was the one with the original need. **Correct numbers are not the same as a useful product**, which is the trap a data team falls into most often, because correctness is the part we know how to check. And **the cost of the missing question was nine days**, which is cheap. Asked in the first interview it would have cost nine minutes.

Tools

No software to install for this chapter.

Job	What to use
Process maps	draw.io (free, browser or desktop), Lucidchart, Visio, or pen and paper first
BPMN specifically	draw.io and Lucidchart both ship the BPMN symbol set
Requirements documents	whatever the company already uses: Word, Google Docs, Confluence
User stories and tracking	Jira, Azure DevOps, Trello, or a shared board. Chapter 26 covers these in practice
Traceability matrix	a spreadsheet, using Chapter 11's skills
Metric definitions	wherever both teams will look: a data dictionary, a BI tool's shared model (Chapter 16), or a dbt project (Chapter 32)
Interview notes	anything, as long as it is dated and kept

Draw the first version of any process map by hand, in the room, with the people who run the process. A tidy diagram invites agreement; a messy one on a whiteboard invites correction, and correction is what you are there for.

The project: map order-to-cash, and specify one data product

Work on Riverstone's order-to-cash process from Chapter 3, section 3.2. Everything you need is in that chapter.

1. Draw the as-is swimlane. Four lanes: customer, sales, warehouse, finance. Place all ten steps. Mark every crossing between lanes. Add at least one exception path that Chapter 3 implies but does not draw, for example what happens when the stock is not actually available at step 5.

- 2. Mark the data damage.** On the same map, mark each handoff where a data quality problem is born, and say in one line what the resulting column looks like when you later query it.
- 3. Write the gap analysis.** Pick three gaps. For each, write the as-is, the to-be, the gap, the **root cause**, and the resulting numbered requirement. One of the three must have a root cause that is different from the obvious one.
- 4. Rank them.** Score your three gaps on frequency, time saved per month, error cost, and build effort. Say which you would do first and why, in two sentences. Use Chapter 3's arithmetic as your model and label your numbers as estimates.
- 5. Specify the winner as one of the four data products** from section 25.6: a report or dashboard, a pipeline, a model, or a metric definition. Write one business requirement, two to four functional requirements, and at least three non-functional ones, of which at least two must come from the data list in section 25.5: freshness, grain, completeness, reconciliation, timeliness, history, access, volume.
- 6. Write two user stories for it**, each with at least three acceptance criteria in Given/When/Then form, including one edge case and one reconciliation criterion.
- 7. Write the UAT plan for one story:** the criteria a user would run, the one complete period you would reconcile and against what, and who signs it off.

You have done this well if somebody who has not read Chapter 3 could take your pack and build the right thing, and if the person who owns the process would recognize their own job in your as-is map.

You've got it when...

- You can say in one sentence how a business analyst's output differs from a data analyst's, a data scientist's, and a data engineer's.
- You can name the six SDLC phases and say what the BA does in each, including the three after sign-off.
- You can explain what a swimlane adds to a flowchart, and why every handoff is also a future data quality problem.
- You never propose a to-be without having drawn the as-is.
- You can take "we need a way to see X" and produce four questions that have to be answered before anything is written down.
- You can tell a business requirement from a functional one from a non-functional one, and you write freshness, grain, completeness, and reconciliation into the first draft rather than the last.
- Given a request, you can say which of the four data products it is, and name the questions that product specifically needs answered.
- You can pick between a BRD, an FRD, and an SRS by naming who will read it.
- You can write a user story with acceptance criteria that include an edge case and a reconciliation check.

- Your gap analysis rows end in numbered requirements and name root causes rather than symptoms.
 - You can explain why QA passing does not mean UAT will, and why a data UAT has to reconcile as well as test.
 - You can look at a process map and point at the steps worth automating, and at the ones that should stay manual.
-

Recap

A business analyst works out what a business needs, writes it down so it can be built, and stays until what was built is what was needed. Four verbs: elicit, analyze, specify, validate. On a data team the BA sits beside the data analyst, the data scientist, and the data engineer, and the four verbs are the part of all four jobs that happens before the code.

The work spans all six phases of the software development life cycle, not only the first. Most of the damage happens in the three phases after sign-off, when the BA has stopped paying attention.

Turn asks into requirements by finding the holes: who and what they will do differently, what the words actually mean, what happens on the exception path, and how often. Map the process before changing it. A flowchart shows what happens; a swimlane shows whose job each step is and exposes the handoffs, which is where delay, error, and bad data all collect; BPMN makes the map readable by people who were not in the room. Always as-is before to-be.

Separate business, functional, and non-functional requirements, number them, and keep the trail from each one to the test that proves it. For data work the non-functional list has its own members: freshness, grain, completeness, reconciliation, timeliness, history, access, and volume. They are the ones that get dropped and then cause the incident.

Almost every request becomes one of four things, and each has its own questions. A **report or dashboard** needs to know who opens it and what they do next. A **pipeline** needs to know the source of truth, what happens when it is late, and what happens when the past changes. A **model** needs to know the decision it serves, the cost of each kind of mistake, and what it is being compared against. A **metric** needs one sentence that two teams will both sign.

Use cases carry whole interactions with their alternative flows; user stories carry slices, and a story without acceptance criteria is not finished. For data, add a reconciliation criterion. Gap analysis compares as-is to to-be, names the root cause, and ends in a requirement. UAT asks whether the specification was right, which QA structurally cannot, and for a data product it also asks whether the numbers agree with something the business already trusts.

For automation, find the re-keying, the copy-paste, the emailed files, the manual matching, and the shadow spreadsheets. Rank on frequency, time, error cost, and effort. Leave the judgment steps alone. And specify a tolerance, an exception route with a named owner, and an audit trail, because those three separate an automation people trust from one they work around.

Practice exercises

Warm-up

1. In one sentence each, say what a business analyst, a data analyst, a data scientist, and a data engineer produce.
2. Name the six phases of the SDLC in order.
3. What does a swimlane diagram show that a plain flowchart does not, and why does that matter to someone who will later query the data?
4. Classify each as business, functional, or non-functional: (a) the dashboard shall open in under five seconds; (b) Riverstone needs to reduce the time between shipping and invoicing; (c) when an order status changes to Shipped, the system shall raise the invoice; (d) one row shall be one order line.

Core

5. Turn this ask into four questions you would need answered before writing a requirement: *"Can we get an alert when a big order comes in?"*
6. Write a user story with four acceptance criteria for: *finance wants to know which invoices are overdue by more than thirty days*. One criterion must cover an edge case and one must be a reconciliation check.
7. Write a full gap analysis row for step 5 of Chapter 3's order journey, the stock check where Neha phones the warehouse. Include a root cause that is not "the warehouse is slow to update".
8. A stakeholder asks for "a dashboard like the one the other team has". Name three things that could go wrong if you write that down as the requirement, and the first question you would ask instead.
9. Riverstone's finance assistant spends about forty minutes a day matching bank lines to invoices, five days a week, forty-eight weeks a year. Estimate the annual hours, and say what else you would need to know before recommending that it be automated.
10. For each of the four data products in section 25.6, write the one non-functional requirement you would refuse to ship without, and say why.

Stretch

11. Write UC-07, a use case for "raise a credit note for a returned delivery", with a main flow and two alternative flows.
12. Write the requirement for a churn model so that a vendor could build it without talking to you. Include what it must beat, and what must never be scored.
13. A dashboard passes every QA test, reconciles to the ERP to the rupee, and fails UAT. Write four sentences you would say in the meeting where that is discovered, in the order you would say them.

Think about it (no calculation needed)

14. The chapter says the as-is map is often the most valuable deliverable before any change is made. Why would that be true, and what does it suggest about how you should run the mapping session?

15. Chapter 58 found an email-intake automation that was right 88% of the time and concluded it should not run unattended. What would you need to know about the other 12% before agreeing?
-

Key terms

business analyst · elicitation · software development life cycle (SDLC) · waterfall · Agile · flowchart · swimlane diagram · handoff · BPMN · as-is · to-be · business requirement · functional requirement · non-functional requirement · freshness · grain · completeness · reconciliation · timeliness · access control · data product · BRD · FRD · SRS · requirements traceability matrix · use case · main flow · alternative flow · user story · acceptance criteria · Given/When/Then · baseline · gap analysis · root cause · user acceptance testing (UAT) · sign-off · re-keying · copy-paste integration · manual matching · shadow system · tolerance · exception path · audit trail

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 26, The Professional Toolkit**, covers Agile, Scrum, Kanban, and Jira as they are actually run, plus Git for the documents and queries this chapter produces.
 - **Chapter 27** turns your Part II projects, including this chapter's requirements pack, into a portfolio.
 - **Chapter 14** cleans the data that the handoffs on your process map created.
 - **Chapter 16** builds the dashboard you specified, on a shared model that holds the metric definition.
 - **Chapters 36 and 39** measure whether a model is good enough, which is the number your requirement has to name in advance.
 - **Chapters 45 and 46** build the pipeline, with the late-data and backfill behavior you specified.
 - **Chapter 47** turns your business rules and non-functional requirements into automated data-quality tests and data contracts.
 - **Chapter 51** builds the system integrations that close the gaps this chapter finds.
 - **Chapter 60** takes non-functional requirements to architecture scale, where they decide the shape of the whole system.
 - **Chapter 63** governs a portfolio of automations once there are more than a few.
 - **Chapter 76B** is the business analyst question bank: requirements, BRD against FRD against SRS, user stories, process mapping, SDLC, gap analysis, UAT, and full worked interview scenarios.
-

Answers to practice exercises

1. A business analyst produces a specification: process maps, requirements, and acceptance criteria precise enough that somebody can build the right thing. A data analyst produces an answer: a number, a chart, or a memo that helps someone decide. A data scientist produces a model and an honest estimate of how well it works. A data engineer produces a pipeline, a table, and a contract that make the data arrive reliably.
2. Requirements, design, build, test, deploy, maintain.
3. Who is responsible for each step, and therefore where work crosses from one team to another. Those crossings, the handoffs, are where delay and error concentrate. They matter to anyone who later queries the data because a handoff is where a human re-types, scans, or matches something, which is exactly where the wrong product code, the missing delivery date, and the payment against the wrong invoice come from.
4. (a) non-functional, it says how well; (b) business, it says why and contains no technology; (c) functional, it says what the system must do; (d) non-functional, it is a grain statement, and it is the one most often left unwritten.
5. Any four of: Who gets the alert, and what will they do when it arrives? What counts as "big", in rupees or in units, and is the threshold the same for every customer? How fast does it have to reach them, and does that change at 11 p.m.? What happens if the order is later cancelled or the amount corrected? How often would this have fired on last year's data? The last is the one that decides whether the feature is useful or noise.
6. For example:

As a finance assistant, **I want** a list of invoices overdue by more than thirty days, **so that** I can chase them before they age further.

AC-1 *Given* today's date, *when* the list is generated, *then* it includes every invoice whose due date is more than thirty days before today and whose outstanding amount is greater than zero. **AC-2** *Given* an invoice that has been part paid, *when* the list is generated, *then* it appears with its remaining balance, not its original total. **AC-3** *Given* an invoice raised against a cancelled order, *when* the list is generated, *then* it does not appear. **AC-4** *Given* the list is generated for a past month-end, *when* its total is compared with the ERP's receivables ageing for that date, *then* the two agree to the rupee.

AC-2 and AC-3 are the edge cases; AC-4 is the reconciliation. A list that shows original totals for part-paid invoices is the classic version of this report being wrong.

7. For example:

As-is. Before confirming stock, the sales executive phones Bhiwandi Main, because the ERP's stock figures are updated once a day from the warehouse's own spreadsheet. **To-be.** Stock figures in the ERP are current enough that a sales executive can trust them when confirming an order. **Gap.** The ERP is not the system of record for stock. A spreadsheet is, and it is reconciled to the ERP once a day. **Root cause.** Not that the warehouse is slow. The warehouse keeps a shadow system because the ERP's stock screen does not fit how they work, so they maintain their own and copy it over. Removing the copy step without fixing the reason the spreadsheet exists will produce a second workaround. **Requirement.** FR-09: stock movements shall be recorded in the ERP at the point of picking, and the ERP shall be the system of record for available stock. **Depends on.** A separate piece of work to find out why the warehouse prefers the spreadsheet, before FR-09 is built.

8. Three risks: the other team's dashboard answers their question, not this stakeholder's; you inherit their metric definitions, which may not match yours, the problem Chapter 23 section 13 describes; and you will be judged against a thing that already exists rather than against the need. First question: "What decision would you make with it that you cannot make today?"

9. Forty minutes a day, five days a week, is 200 minutes a week, about 3.33 hours. Over forty-eight weeks that is about 160 hours a year, roughly four working weeks. Before recommending automation you would want: what share of lines match cleanly today, because that decides how much of the 160 hours actually goes away; what the tolerance for a match should be; what the errors cost, including customers chased for money they have paid; who handles the exceptions once a machine handles the rest; and what it would cost to build.

10. For example. **Dashboard:** reconciliation, because a dashboard whose numbers are not tied to a source the business trusts will be checked against a spreadsheet forever. **Pipeline:** timeliness with a named alert owner, because a pipeline that fails silently is worse than no pipeline. **Model:** the baseline it must beat, because without it "84% accurate" cannot be judged. **Metric:** that the definition is recorded once and referenced, because a definition that lives in a query gets reinvented by every new report.

11. For example:

UC-07: Raise a credit note for a returned delivery Actor: finance assistant. **Precondition:** an invoiced order has a recorded return. **Main flow:** 1. The assistant opens the return. 2. The system shows the original invoice and its lines. 3. The assistant selects the returned lines and quantities. 4. The system calculates the credit at the original prices and discounts and raises a credit note against the invoice. 5. The invoice's outstanding balance reduces by the credit. **Alternative A, invoice already paid in full:** at step 4 the system creates the credit as an unallocated balance on the customer's account and flags it for refund or offset. **Alternative B, partial return of a discounted line:** at step 4 the discount is applied pro rata to the returned quantity, and the system shows the calculation before the assistant confirms. **Postcondition:** a credit note exists, linked to the original invoice, and the customer's balance reflects it.

12. For example: **BR**, Riverstone needs to know which accounts are likely to stop ordering, early enough to act. **FR-01**, produce for each active account a probability of no order in the next ninety days. **FR-02**, write the output weekly to a table carrying the score date and the model version. **FR-03**, carry with each score the three features that contributed most to it. **NFR-01**, the model shall beat the rule "no order in ninety days" on accounts held back from training, measured on the metric agreed in BR. **NFR-02**, no score shall be produced for an account with fewer than three historical orders. Without NFR-01 the vendor can deliver anything and call it a model; without NFR-02 you will get confident scores for accounts with no history.

13. For example: "The build does what the specification says and the numbers reconcile, so this is not a defect." / "The specification is wrong, and that is on me: I never asked what you would do differently once you had it." / "What I am hearing is that the decision you need to make is which quiet account to spend time on, which needs a reason and a value, not a list of names." / "Give me two days to rewrite the requirement and come back with what that costs."

Owning it first is not politeness. It moves the meeting from blame to scope in one sentence.

14. Because the people who run a process usually know only their own step and the two next to it, so the end-to-end picture is new information to everybody in the room, including the person who owns the process. It suggests you should run the session with the people who do the work rather than their managers, draw it roughly and in public so it invites correction, and expect the argument in the room to be more valuable than the diagram you leave with.

15. What the 12% consists of, and how visible it is. Twelve percent that fails loudly, where the system says it could not read the email, is a manageable exception route. Twelve percent that fails quietly, where the system reads the email wrongly and books a plausible but incorrect order, is a different thing: it produces errors that look like data. You would also want to know the cost of one silent error, whether the failures cluster in one customer or format, and whether a person downstream would catch them.